perplexity

Show me a simple code example using the OpenAI Agents SDK

Here's the simplest working Python example with the OpenAI Agents SDK: install the package, set your API key, define an agent, and run it. [1] [2]

1. Install and set key

```
pip install openai-agents
export OPENAI_API_KEY="sk-..."
```

The SDK's Python package is `openai-agents`, and it reads `OPENAI_API_KEY` from your environment. [2] [1]

2. Minimal agent

Create `agent.py`:

```
import asyncio
from agents import Agent, Runner

agent = Agent(
    name="History tutor",
    instructions="You answer history questions clearly and concisely.",
    model="gpt-6-astra",
)

async def main():
    result = await Runner.run(
        agent,
        "When did the Roman Empire fall?"
    )
    print(result.final_output)

if __name__ == "__main__":
    asyncio.run(main())
```

Run it:

```
python agent.py
```

This defines one agent with a name and instructions, sends one user request, and prints the agent's final answer. [2]

3. Add a simple tool

This version gives the agent a Python function it can call:

```
import asyncio
from agents import Agent, Runner, function_tool

@function_tool
def history_fun_fact() -> str:
    """Return a short history fact."""
    return "Sharks are older than trees."

agent = Agent(
    name="History tutor",
    instructions=(
        "Answer history questions clearly. "
        "Use history_fun_fact when it helps."
    ),
    model="gpt-6-astra",
    tools=[history_fun_fact],
)

async def main():
    result = await Runner.run(
        agent,
        "Tell me something surprising about ancient life on Earth."
    )
    print(result.final_output)

if __name__ == "__main__":
    asyncio.run(main())
```

The `@function_tool` decorator turns a normal Python function into a tool; the SDK generates the tool schema and lets the agent decide when to invoke it. [1] [2]

What happens

1. Your code creates an `Agent` with instructions and optional tools.
2. `Runner.run()` starts the agent loop.
3. If needed, the model calls `history_fun_fact`.
4. The SDK executes the Python function and feeds the result back to the model.
5. `result.final_output` contains the completed answer. [1] [2]

For a first project, keep the instructions narrow and add only one tool at a time; the SDK also supports handoffs to specialist agents, guardrails, sessions, and tracing once your basic agent works. [2] [1]

⁂

1. https://openai.github.io/openai-agents-python/
2. https://developers.openai.com/api/docs/guides/agents/quickstart